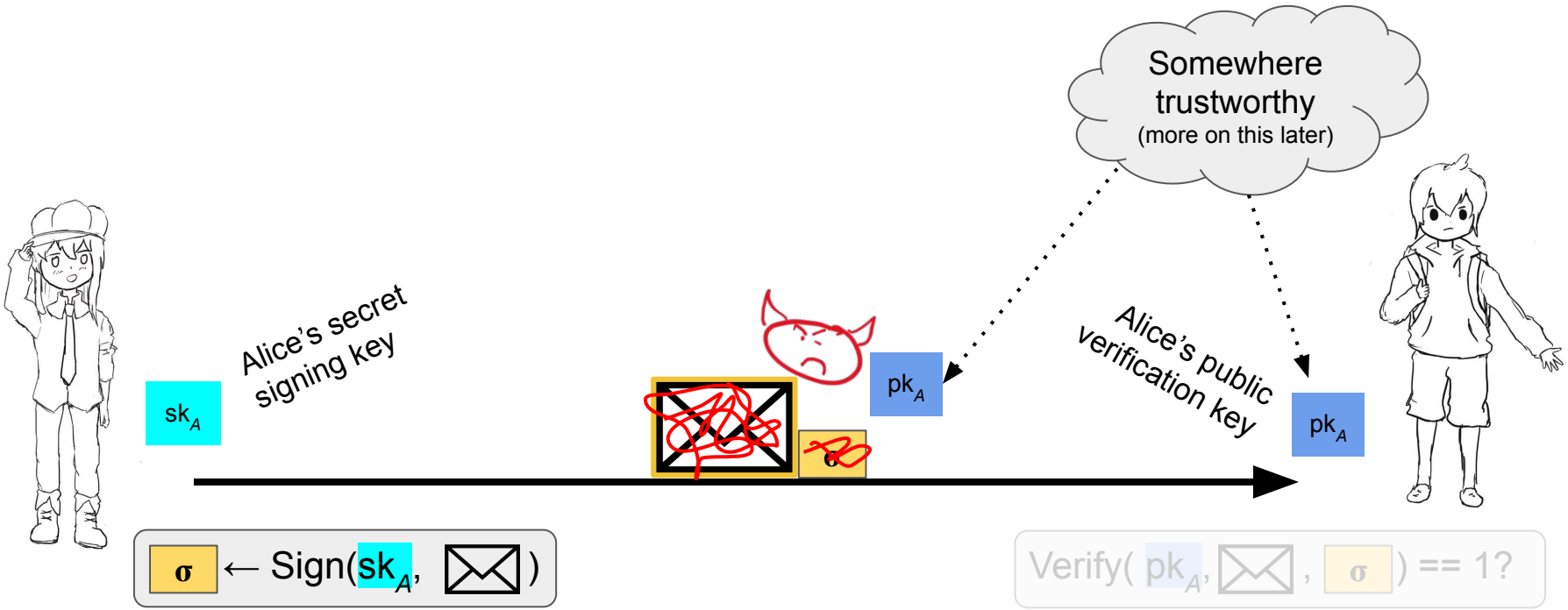
Randomness and Pseudorandom Generators

CS 161 Spring 2026 - Lecture 15

Last time: Digital signatures

- A public-key integrity primitive
  - Alice signs a message with her secret key
  - Anyone can verify that Alice signed // Using Alice's public verification key
- Signatures are used <u>everywhere</u> on the internet
  - SSH, TLS, git commits, Linux packages, …
- There are many constructions of digital signatures
  - RSA signatures are built from the RSA trapdoor one-way permutation

Solution: Digital signatures



Bad news: There is no “best” signature scheme

	Assumption type	Ver. key size	Sig. size	Sign()/sec	Verify()/sec	PQ secure?
ECDSA-256 (also: EdDSA)	Discrete log	32 b	64 b	77,000	25,000	N
RSA-2048	Factoring	256 b	256 b	2,300	90,000	N
SPHINCS+ 128	One-way fn (Hash)	32 b	800 b	5	750	Y
MLDSA-44	Lattice	1,312 b	2,420 b	9,000	25,000	Y
And there are many more…						

Quantum attacks on crypto: One-slide summary

Secret-key cryptography

PRF, Block Cipher, Encryption, MACs, Hash functions

- Largely unaffected by quantum computers
- Brute-force search for *n*-bit decryption key
  - Classically: **2^n time** Quantumly: **$2^{n/2}$ time** ("Grover search")

⇒ Doubling size of keys protects against quantum attacks. (AES-256 and SHA-512 still fine.)

Public-key cryptography

Key exchange, pub-key encryption, signatures

- Quantum attacks totally break all widely used pub-key systems
  - Can recover the secret key from public key in quantum poly time ("Shor's algorithm")
- Vulnerable: Anything based on discrete log or factoring
  - : ~~Diffie-Hellman~~, ~~Elliptic-curve Diffie-Hellman~~, ~~DSA~~, ~~ECDSA~~, ~~RSA~~, …

⇒ Need totally new algorithms. US Gov't is driving transition.

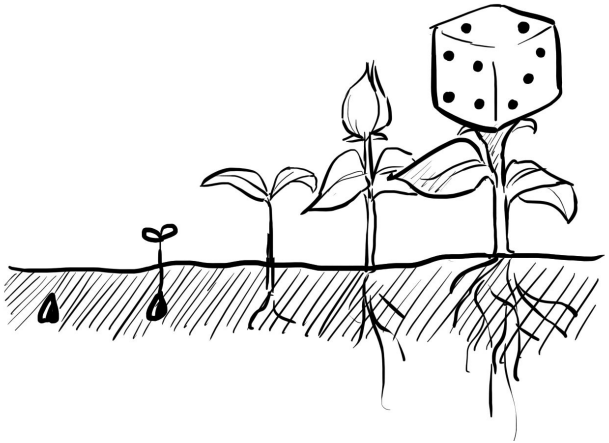
Cryptography roadmap

	Secret-key crypto	**Public-key crypto**
Encryption	✅ One-time pad ✅ CPA security ✅ Counter-mode encryption ✅ CBC encryption ✅ Authenticated encryption	✅ Key exchange (e.g., Diffie-Hellman) ✅ Public-key encryption (e.g., ElGamal)
Authentication	✅ Message auth. codes ("MACs")	✅ Digital signatures (e.g. RSA sigs)

Building blocks	✅ Pseudorandom functions ✅ Block ciphers (e.g., AES) ✅ Hash functions (e.g., SHA2)	✅ Discrete-log problem



Randomness



Textbook Chapter 9

Randomness is essential

The security of our cryptosystems is premised on the randomness of:

- **Secret keys**
  - Encryption
  - MAC
  - Diffie-Hellman
  - ElGamal encryption
- **Nonces / Initialization Vectors (IVs)**
  - Randomized counter-mode encryption
  - CBC-mode encryption
  - Authenticated encryption
  - Digital signatures: DSA, ECDSA, and lattice-based signatures

What is randomness?

- The kind of randomness we need for cryptography:

  “Sample a 128-bit string uniformly at random.”

- What matters is the adversary’s uncertainty about which value we picked
  - Can quantify this with “entropy”
- You can’t tell whether an output is “random” just by looking at it
  - What matters is the process that produced the output
- **Example:** If I flip a coin and tell you it came up “Heads,” can you tell whether it was a fair coin?

Randomness is the source of many security issues

Challenge 1: Randomness is difficult to test for.

- Randomness bugs almost never affect **correctness**
  - All of the cryptosystems we've seen have correctness if you fix the keys and IVs
- How do you test that `ssh-keygen` is producing random SSH keys?
  - Run it 100 times, make sure it never generates the same key twice?
  - Or should you run it 1000 times? Or 10,000 times?
- Common issue: `secret_key = Hash(get_time_of_day())`
  - What matters is the adversary's uncertainty! Time is easy to guess.
  - This code will pass all of your statistical tests

Challenge 2: Randomness failures are catastrophic

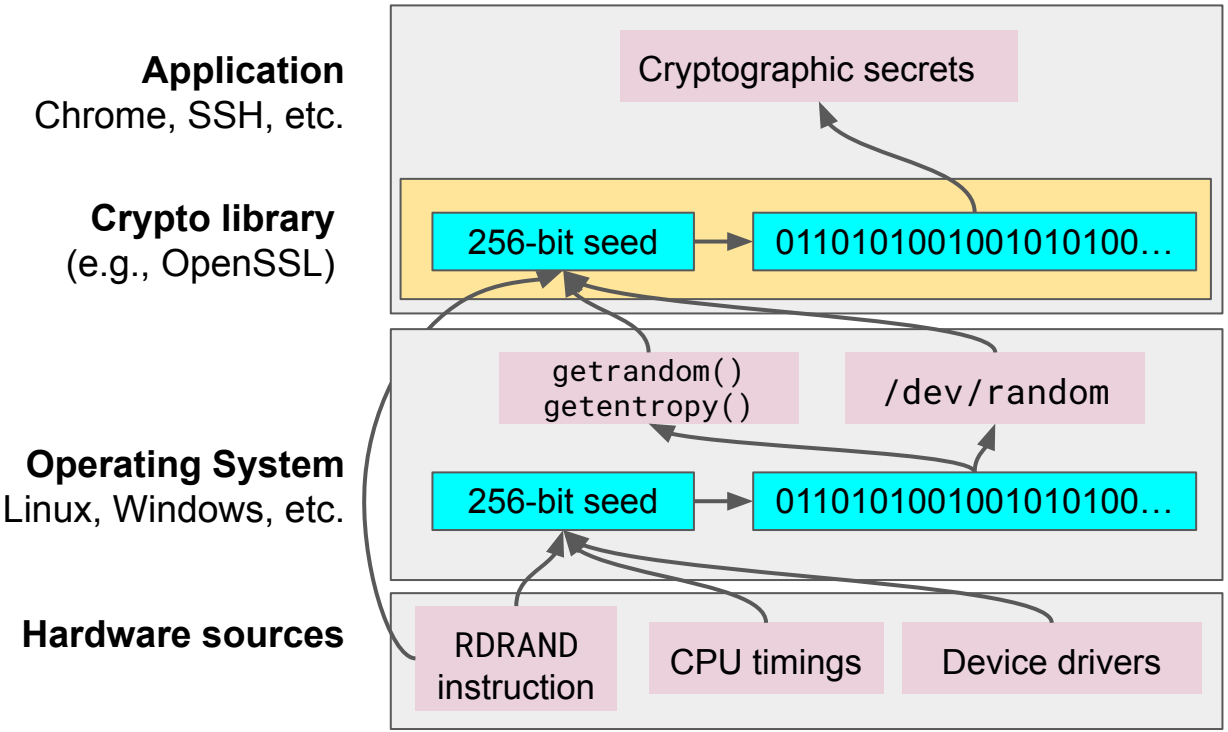
Consequences of using bad randomness

- If you generate your keys using a poor randomness source, attacker may be able to recover your keys
  - E.g., if you use `Hash(time())` as your secret key
  - Attacker guesses time and computes your key

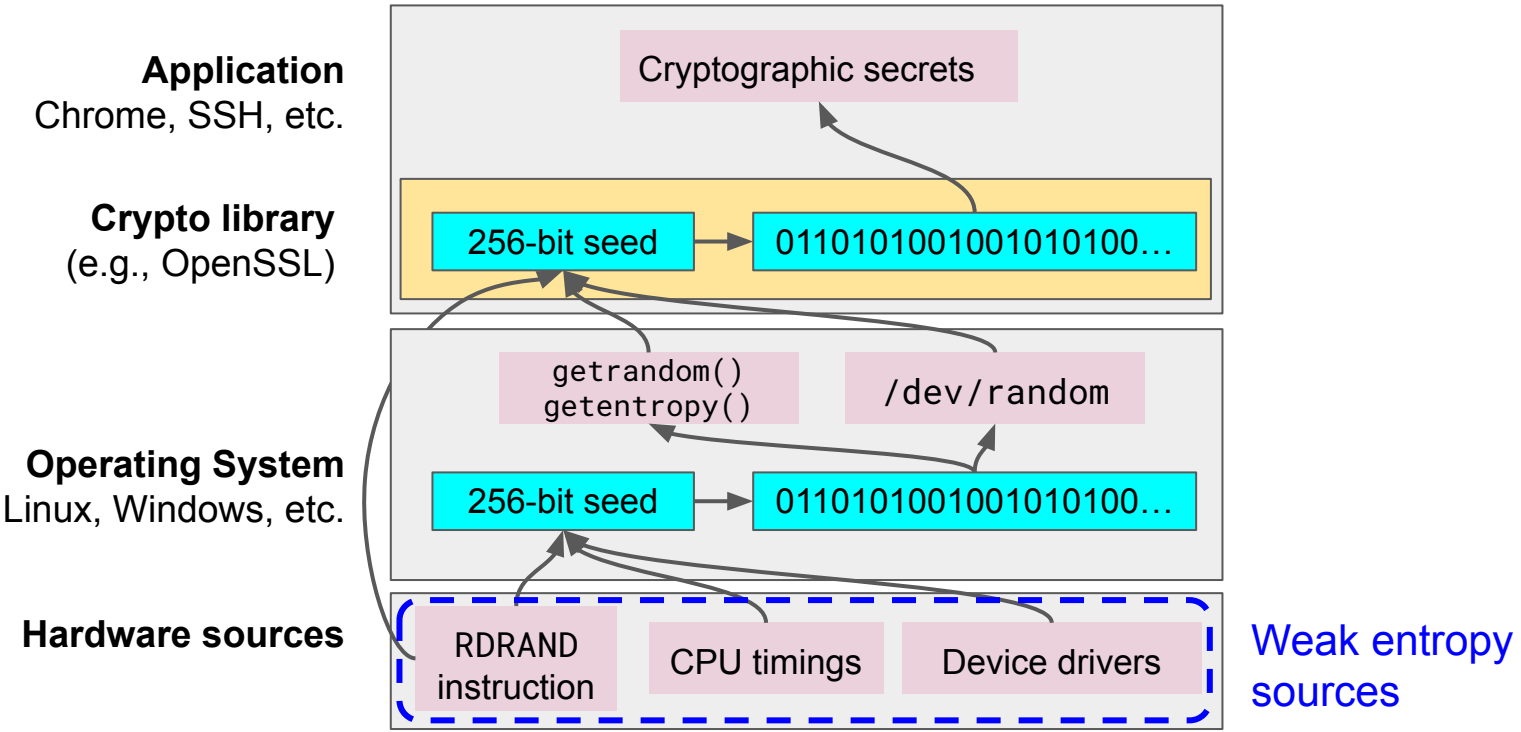
- The DSA/EC-DSA signature schemes fail in a more bizarre way
  - The signature algorithm is randomized
  - For each signature, the signer chooses a *nonce*; the signature includes *f*(*nonce*)
  - Attacker obtains two signatures with same nonce ⇒ **Learns secret signing key!**
  - Attacker learns nonce ⇒ **Learns secret signing key!**
  - This is one of many reasons to dislike DSA/ECDSA

- Nonce reuse with DSA has caused many real-world failures

How computers generate random numbers



Randomness failures



2008: Weak keys in embedded devices

- OS collects randomness from the environment
  - Keypress timings
  - Mouse movements
  - Disk-read timings
  - Network-packet timings

- **Problem:** Network router has no mouse, keyboard, disk
  - Even worse, these devices generate their cryptographic keys on first boot!
  - Many devices had exactly the same key

⇒ In 2008, researchers were able to recover private keys for 0.5% of TLS hosts

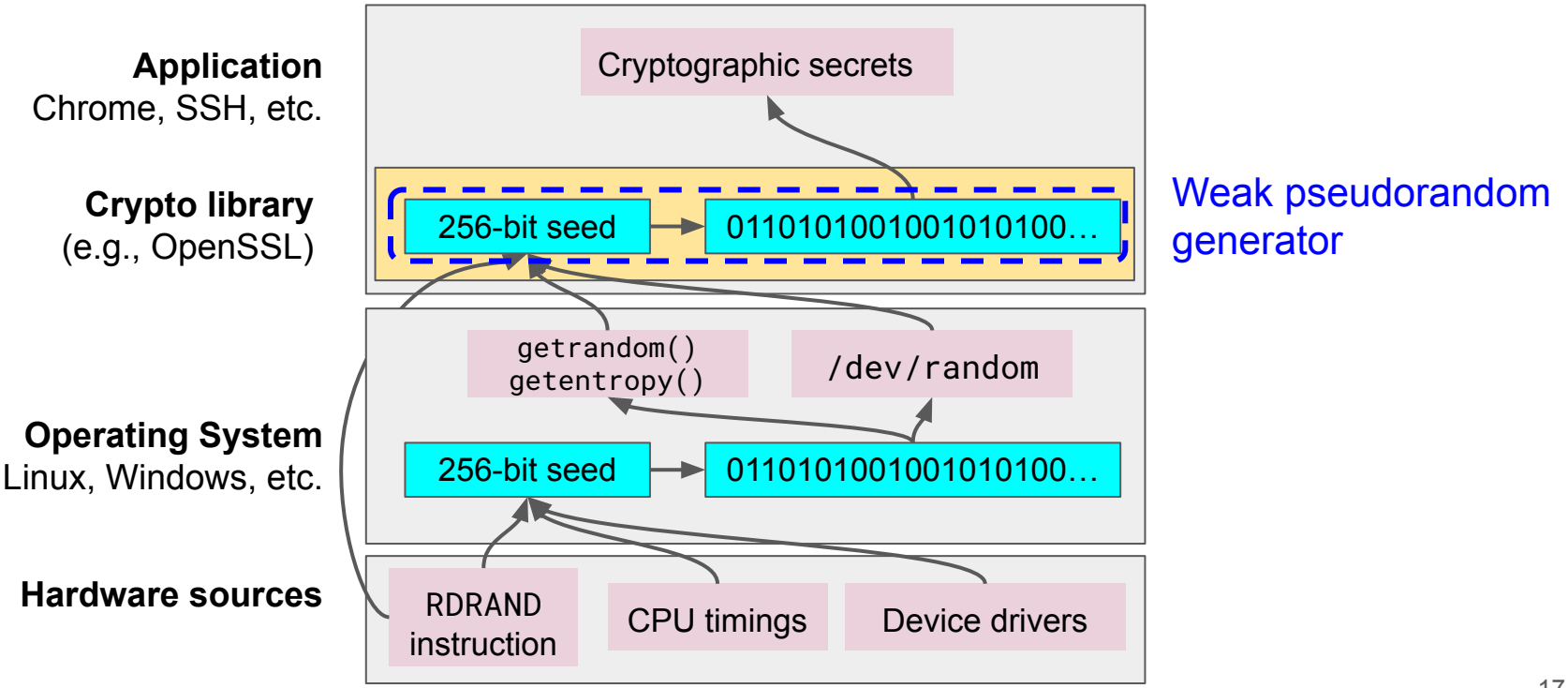
Fix: Gather entropy from more diverse sources,
Gather more (i.e., wait longer) before giving it to applications

2019: Chromebook U2F bug

- Chromebooks have a built-in U2F (universal second factor) security key
  - Uses signatures to authenticate the user
  - Signature algorithm: 256-bit elliptic-curve DSA

- There was a bug in the hardware
  - Instead of using 256-bit nonce, bug caused the nonce to be 64 bits long
  - An attacker with a signature could simply try all possible nonce values to recover it
  - Recovering nonce ⇒ Leaks secret key

Randomness failures



Pseudorandom generator (PRG)

- Stretches a short truly **random seed** into a long **pseudorandom output**

  $G : \{0,1\}^n \rightarrow \{0,1\}^{big}$

- A block cipher or pseudorandom function in counter mode does this!
  - Say we have a PRF F with an n-bit key
  - Can stretch the n key bits into a gigantic number of pseudorandom bits

- OS PRNGs support reseeding too. Not important for our discussion.

Not all PRGs are cryptographic

- Cryptographic security = No efficient adversaries can distinguish
  - Output of *G*(*seed*) on random seed
  - A truly random bitstring of the same length

- PRGs built from cryptographic tools (AES, SHA, etc.) satisfy this

- There are much faster PRGs that don't satisfy this security notion
  - If adversary can see ~100 bytes of output, adversary can predict all future outputs
  - Still useful randomized data structures and randomized algos

- Many library randomness sources are outputs of **NON**-cryptographic PRGs
  - Examples: Python `random`, C's `rand()`, …
  - **Common error:** Using these sources to generate cryptographic secrets. 💀

How a Slot Machine RNG (Ra

cachecreek.com/slot-machine-rng

Incognito

CACHE CREEK CASINO RESORT

HOTEL GAMING PROMOTIONS ENTERTAINMENT RESORT DINING TRANSPORTATION MYCACHECREEK

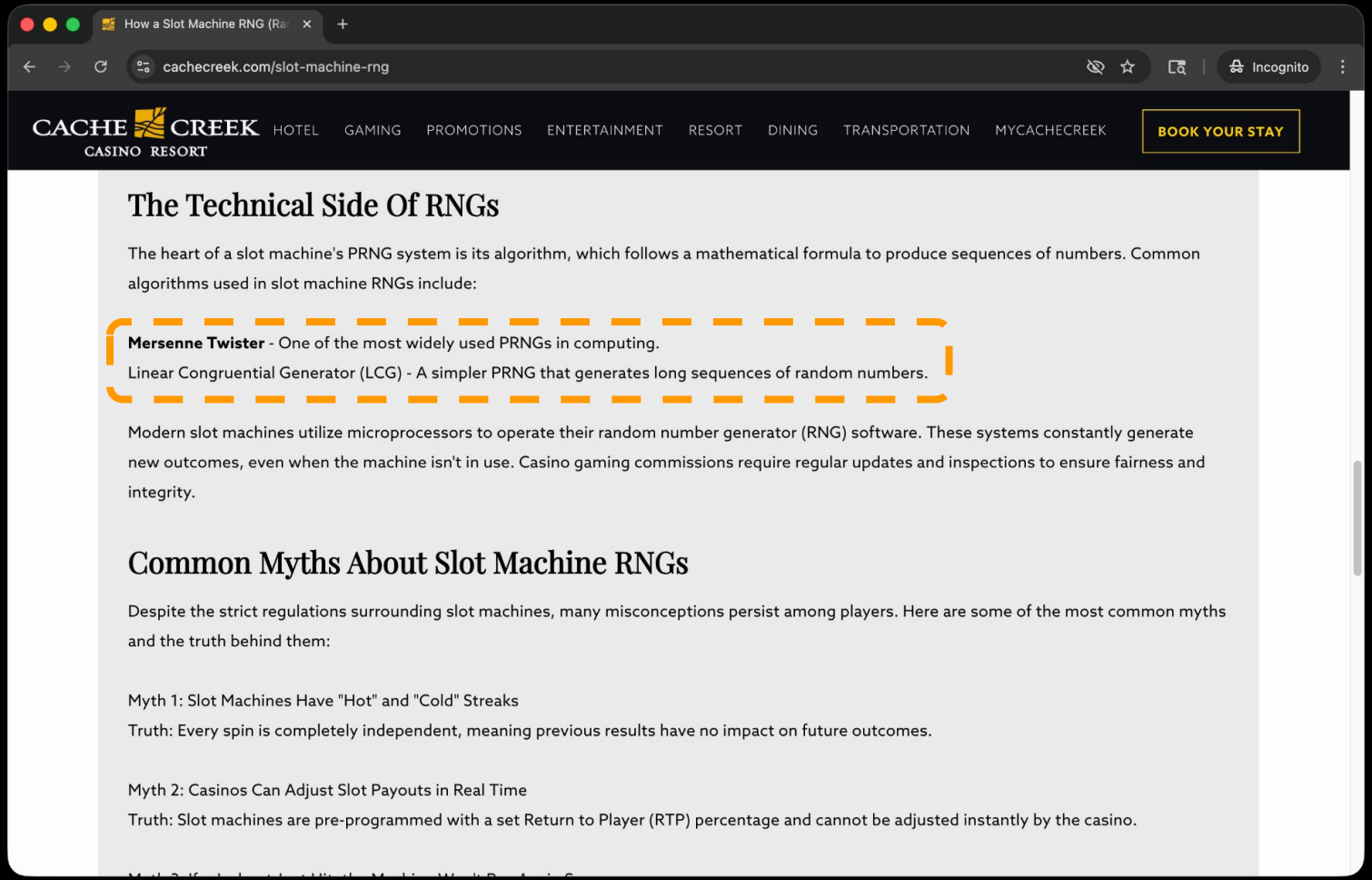
BOOK YOUR STAY



Slot Machine RNG: How It Works

Slot machines are among the most popular and thrilling games in any casino. Whether you're spinning the reels in a physical casino or playing online, the fairness and unpredictability of slot machines rely on one crucial component: the Random Number Generator (RNG).

Understanding how RNGs work can help players make informed decisions, dispel common myths, and enhance their casino experience. In this guide, we'll break down everything you need to know about slot machine RNGs, including how they ensure fairness, the technology behind them, and frequently asked questions.



The Technical Side Of RNGs

The heart of a slot machine's PRNG system is its algorithm, which follows a mathematical formula to produce sequences of numbers. Common algorithms used in slot machine RNGs include:

Mersenne Twister - One of the most widely used PRNGs in computing.
Linear Congruential Generator (LCG) - A simpler PRNG that generates long sequences of random numbers.

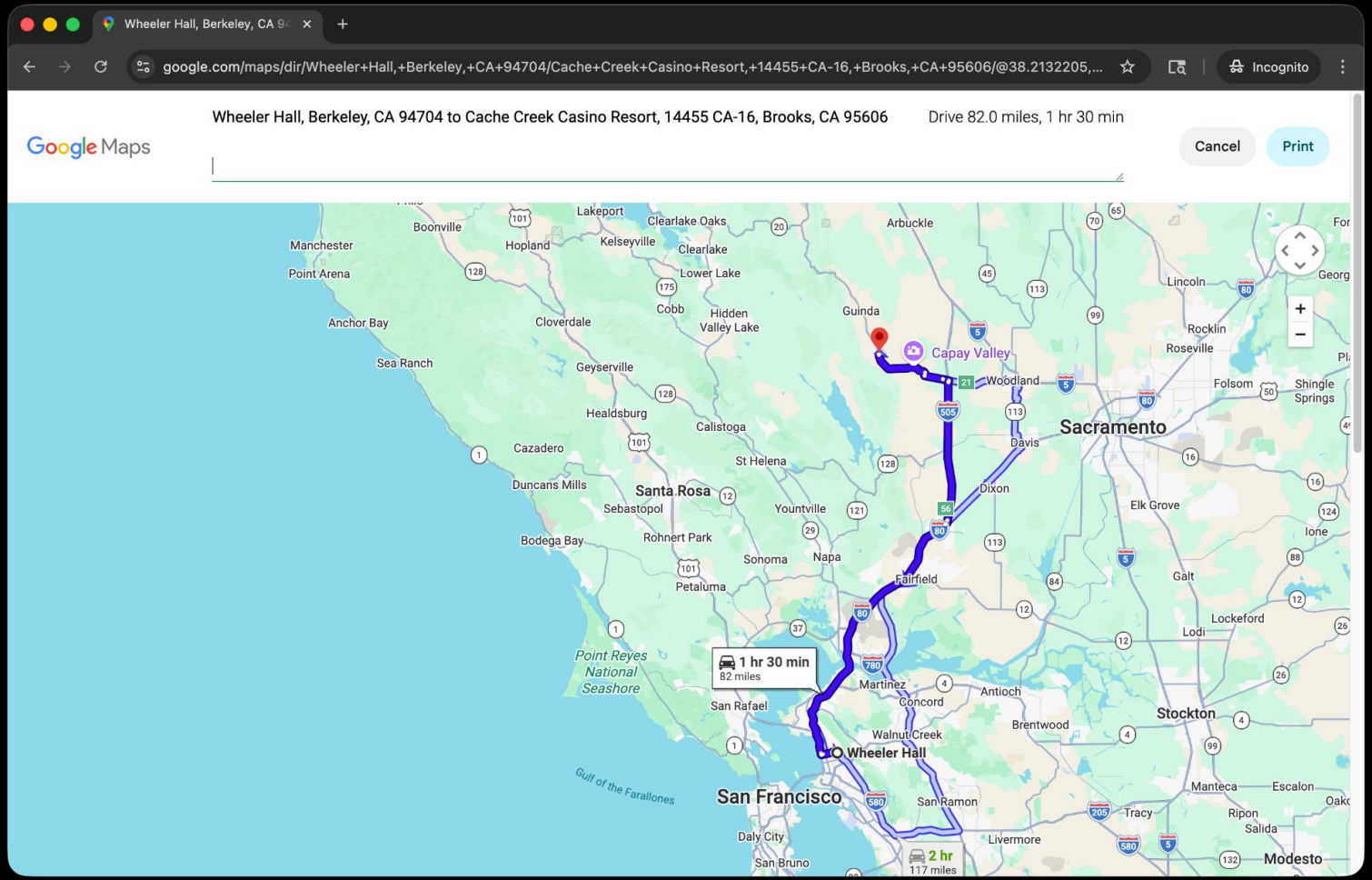
Modern slot machines utilize microprocessors to operate their random number generator (RNG) software. These systems constantly generate new outcomes, even when the machine isn't in use. Casino gaming commissions require regular updates and inspections to ensure fairness and integrity.

Common Myths About Slot Machine RNGs

Despite the strict regulations surrounding slot machines, many misconceptions persist among players. Here are some of the most common myths and the truth behind them:

Myth 1: Slot Machines Have "Hot" and "Cold" Streaks
Truth: Every spin is completely independent, meaning previous results have no impact on future outcomes.

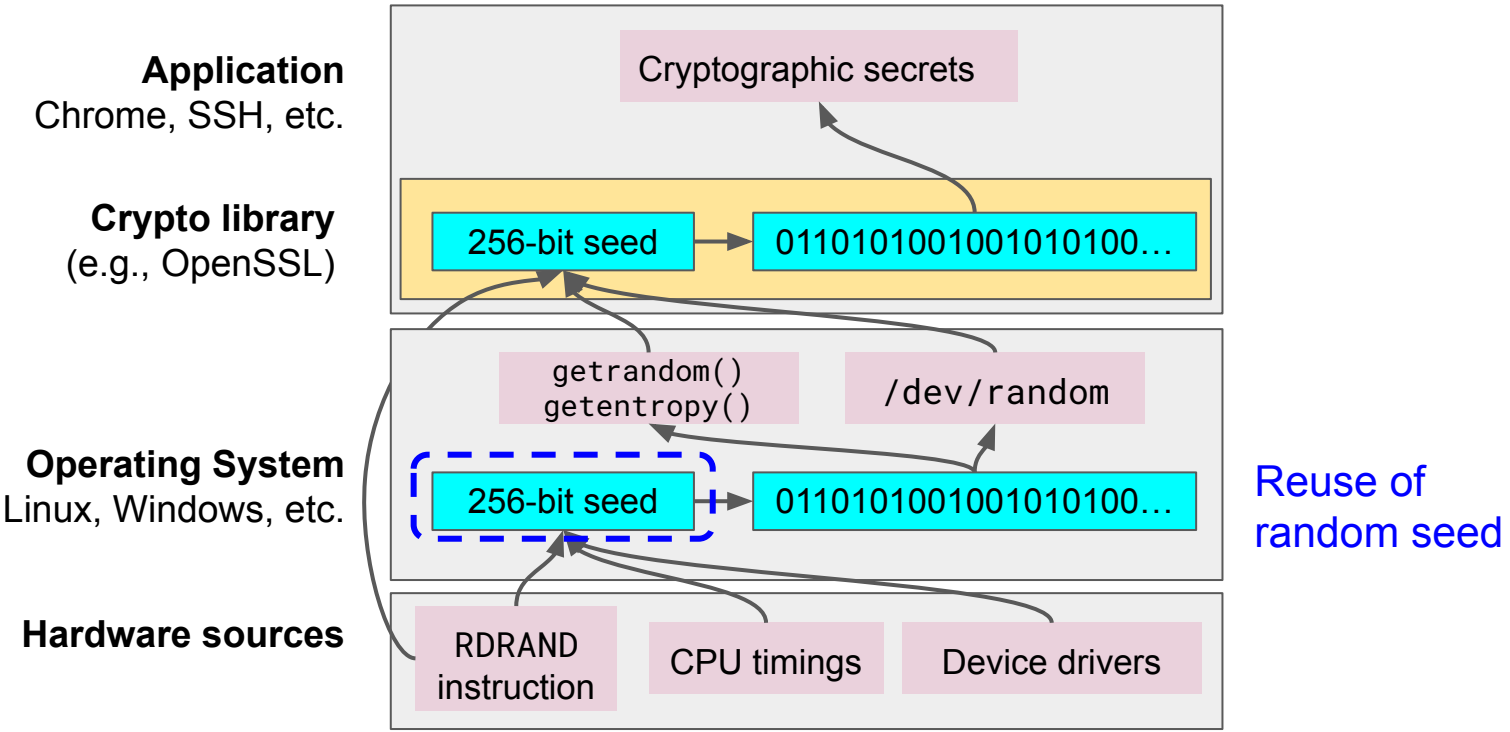
Myth 2: Casinos Can Adjust Slot Payouts in Real Time
Truth: Slot machines are pre-programmed with a set Return to Player (RTP) percentage and cannot be adjusted instantly by the casino.


Wheeler Hall, Berkeley, CA 94
google.com/maps/dir/Wheeler+Hall,+Berkeley,+CA+94704/Cache+Creek+Casino+Resort,+14455+CA-16,+Brooks,+CA+95606/@38.2132205,...
Incognito
Google Maps
Wheeler Hall, Berkeley, CA 94704 to Cache Creek Casino Resort, 14455 CA-16, Brooks, CA 95606
Drive 82.0 miles, 1 hr 30 min
Cancel
Print
Capay Valley
Sacramento
Santa Rosa
San Francisco
Wheeler Hall
1 hr 30 min
82 miles
2 hr
117 miles
Point Reyes National Seashore
Gulf of the Farallones
Stockton
Modesto

2017: Slot-machine hacking

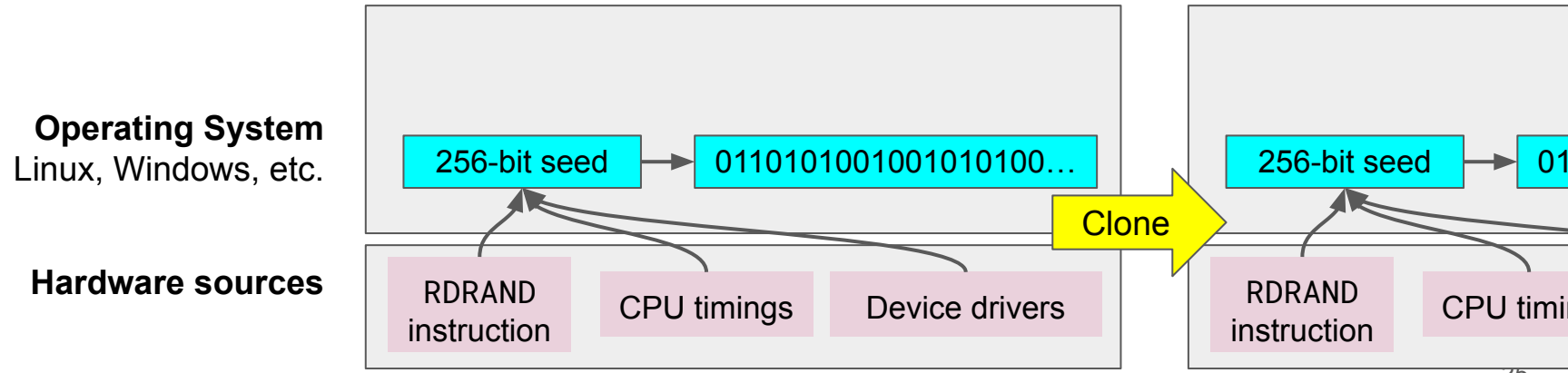
- A casino in St. Louis experienced unusual bad “luck”
  - Suspicious players would hover over the lever and then spin at a specific time to win
- Vulnerability: Slot machines used non-cryptographic PRNGs
  - Given output of a long-enough string of spins, could predict future outputs
- Strategy:
  - Use a smartphone to record results of spins
  - Send recording to mothership
  - Mothership runs computation to predict PRG outputs
  - Alerts you to when to pull the lever for the best chance of winning

Randomness failures

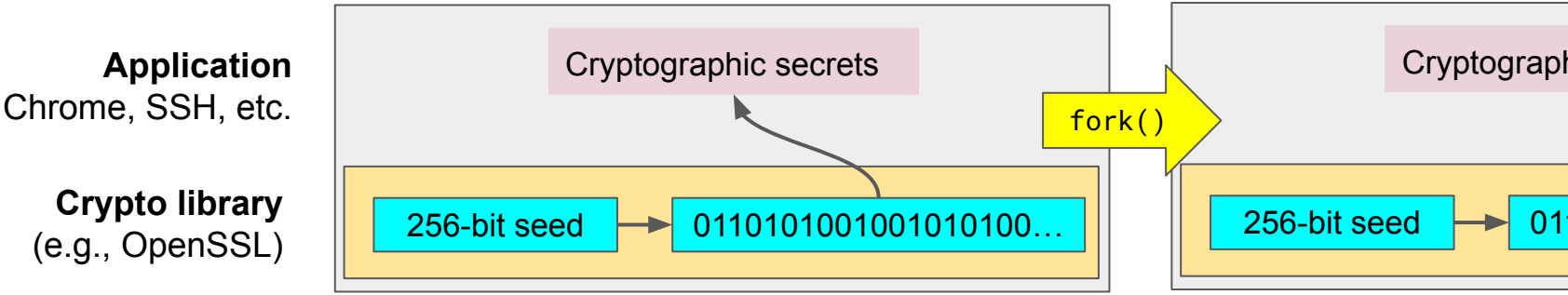


Virtual machines and randomness

- VM = A way to run multiple operating systems on one machine
  - If you used a “Local setup” for Project 1, you were using VMs
- Another convenient thing: Can clone a VM and spin up many copies
  - If not careful: Both clones can have an OS PRG seeded with **exactly the same state**
  - Two different customers’ VMs using same randomness → **Key compromise**

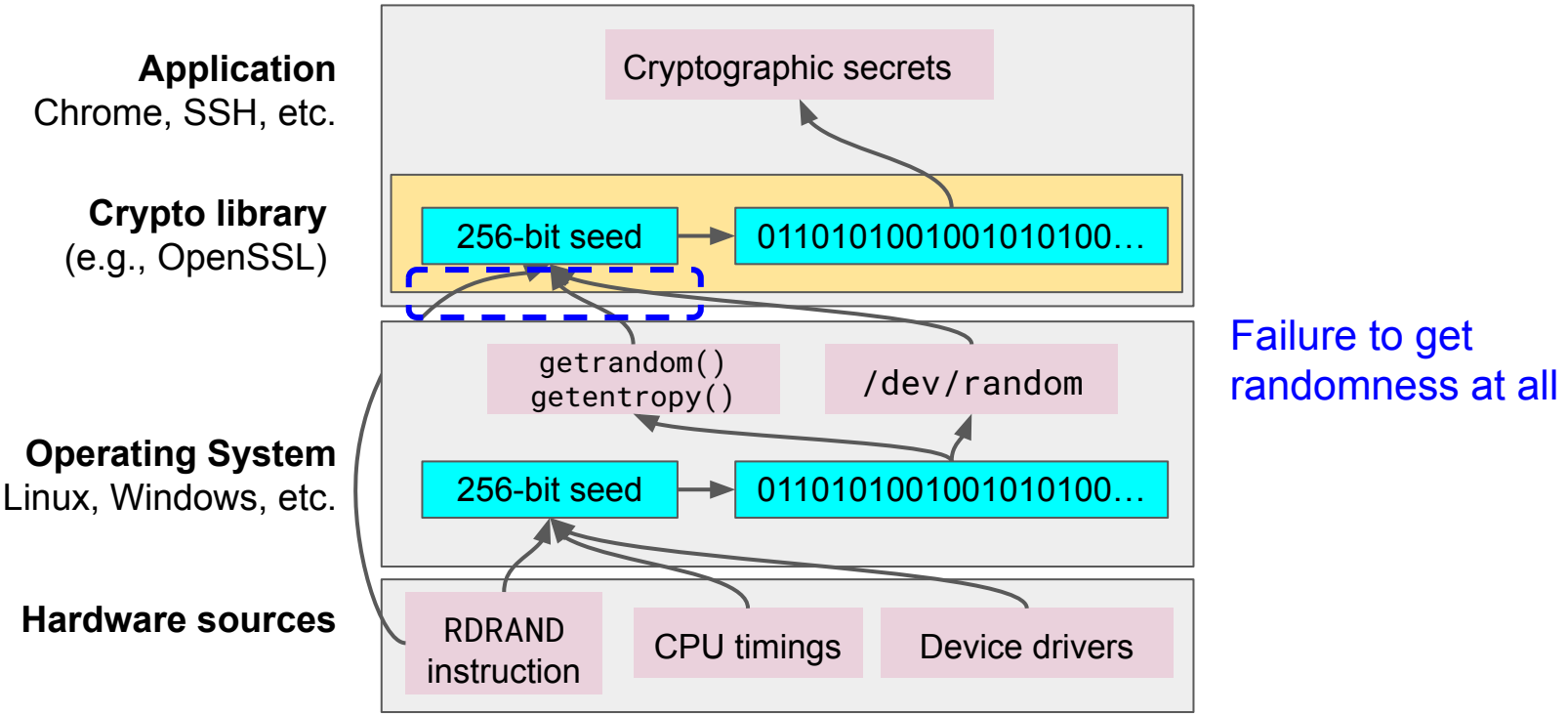


Fork() and randomness



- Very similar issue can happen at the application layer
- `fork()` in Linux/Unix-like OSes copies an entire application process
  - A server process might fork() itself to serve client requests
- Need to be careful to reseed after fork()

Randomness failures



2008: Debian Bug

CVE-2008-0166

- Debian = A popular Linux distribution
  - Debian package manager ships a version of OpenSSL, a critical cryptography library
- OpenSSL generated a PRG seed by hashing:
  - 1) Process ID
  - 2) Randomness from /dev/random + some possibly-uninitialized memory
- Valgrind tool complained about uninitialized memory
  - Debian package maintainer removed source (2) (Debian "bug" #363516)
  - The cleanup caused the PRNG to only be seeded with the process ID
  - There are only 2^{15} (32,768) possible process IDs

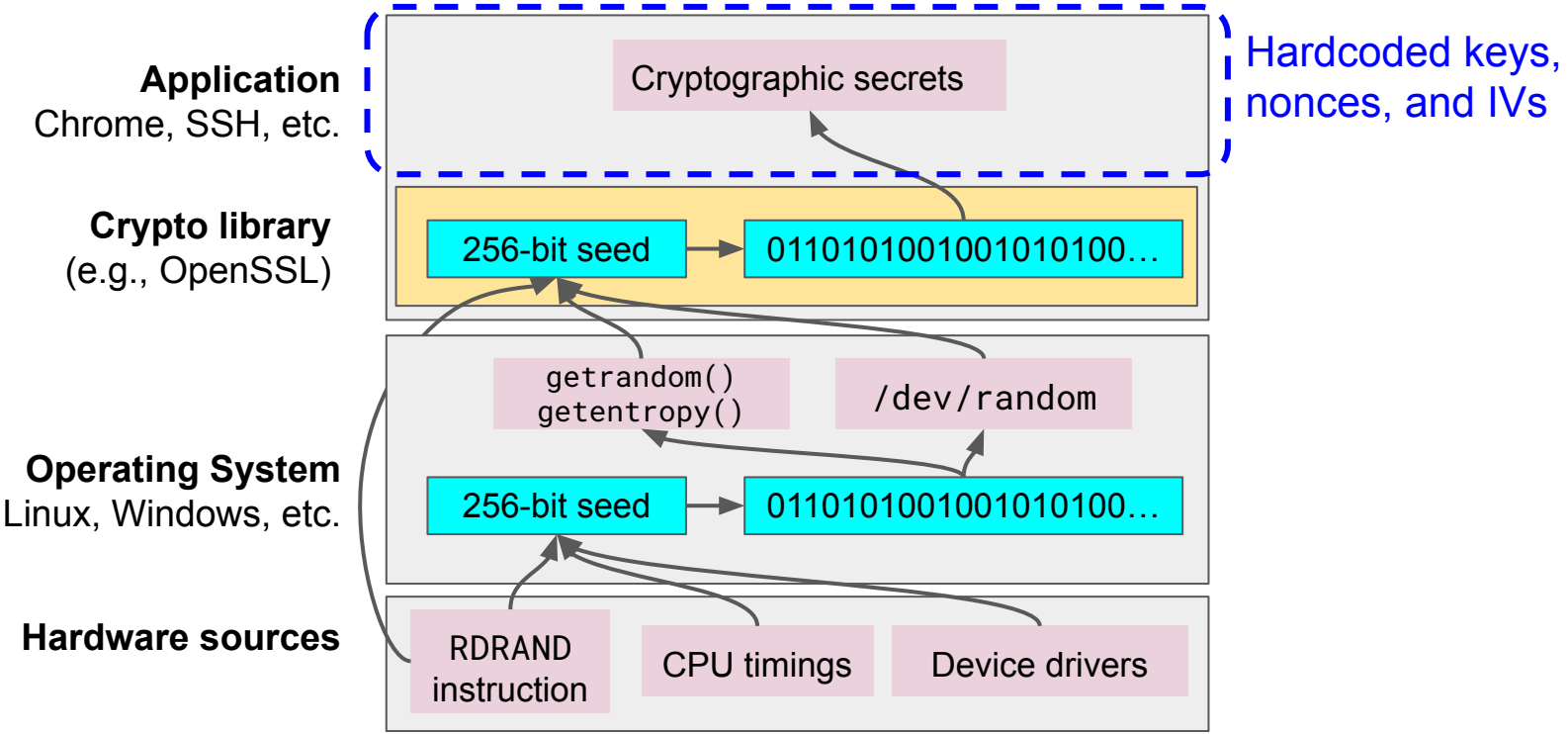
⇒ **Impact:** Easy to deduce private keys generated with the PRNG

  - Set the PRNG to each possible starting state and generate a private/public key pair from each
  - See if the matching public key is anywhere on the Internet

2013: Android SecureRandom vulnerability

- Java Cryptography Architecture provides a `SecureRandom` function
  - Should return output of cryptographically strong PRG, seeded with /dev/random
  - Instead, seed was picked from a space of size $< 2^{64}$
- Multiple Bitcoin wallet apps on Android were affected
  - Bitcoin payments are signed with elliptic-curve DSA and published publicly
  - Insecure RNG caused multiple payments to be signed with the same *nonce*
- Attack: Someone scanned for all Bitcoin transactions signed insecurely
  - Recall: When multiple signatures use the same *nonce*, the attacker can the private key
  - In Bitcoin, your private key unlocks access to all your money

Randomness failures



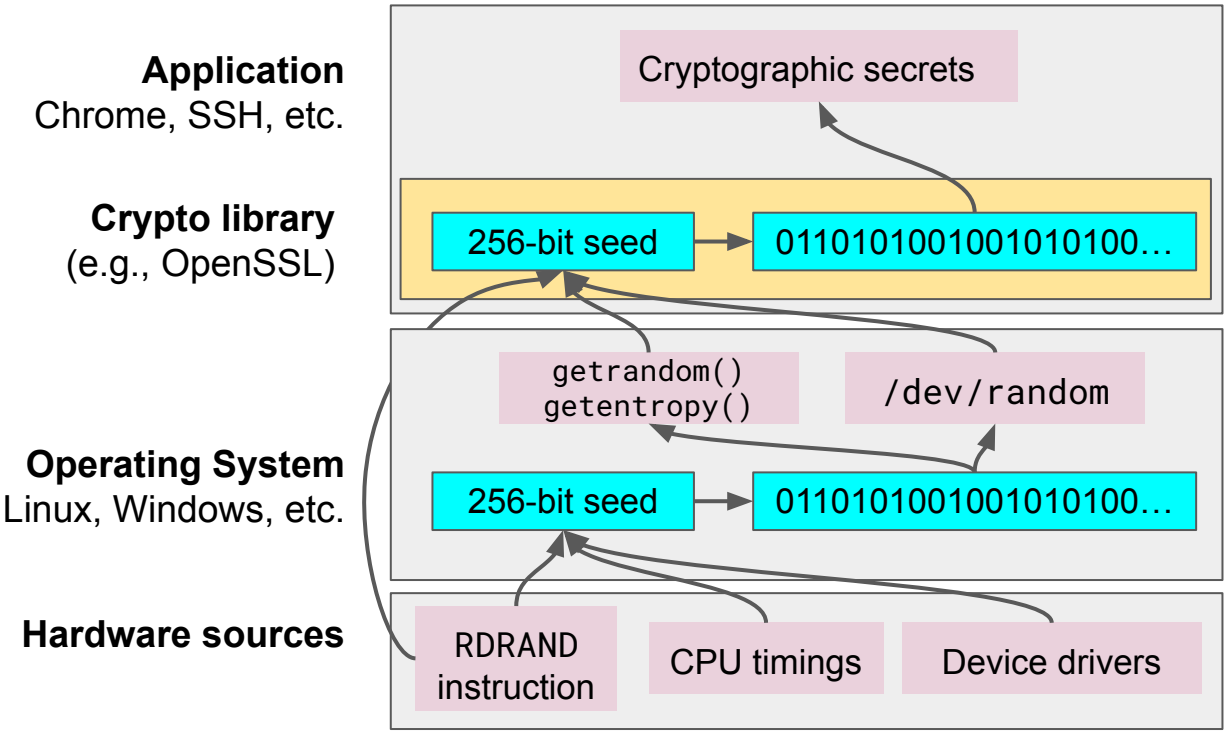
Hardcoded keys, IVs, and nonces

- Surprisingly common failure mode:
  - Hardcoded (fixed, not random) values for keys, IVs, and nonces

Example: Sony PlayStation 3 (2010)

- Originally, Sony let people install Linux on PS3
- Later, Sony added DRM to prevent owners from loading their own OS
  - PS3 would only run Sony-signed software
  - For code signing, Sony used ECDSA
- **Problem:** Sony signed using a fixed ECDSA nonce
  - ⇒ Researchers recovered Sony's secret signing key!

Bottom line: There are many places things can go wrong!



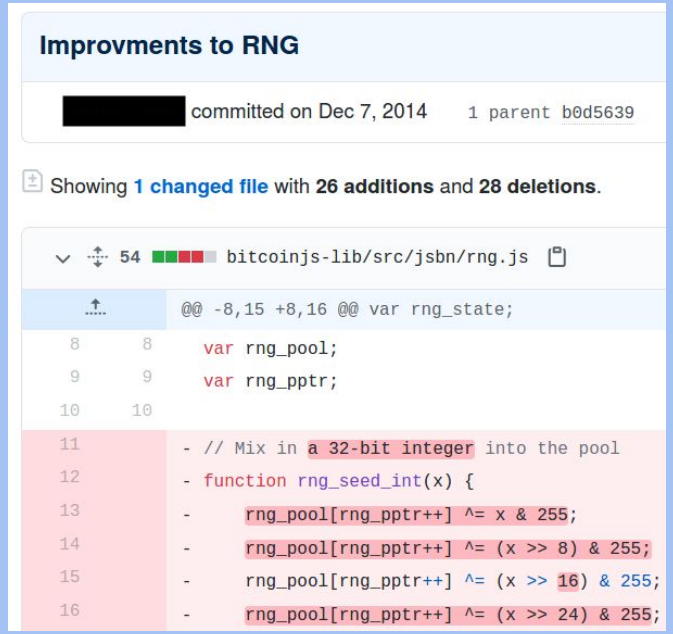
Summary: Randomness

- Randomness failures are difficult to detect
  - Catastrophic when they happen
- **Do:** Use a cryptographically secure PRG when you need really random #s
  - Unique user IDs
  - One-time use tokens
  - …
- **Do:** Read bytes from `/dev/random` to seed your PRG
  - Or better: Use a high-level crypto library that does this for you

Next time: Web security

Breaking Bitcoin Wallets

- What happens if we use a poor source of entropy?
- Bitcoin users use a randomly-generated private key to access their account (and money)
  - An attacker who learns the key can access the money
  - We'll learn more about Bitcoin later
- An "improvment" [sic] to the algorithm reduced the entropy used to generate the private keys
  - Any private key created with this "improvment" could be brute-forced



Improvments to RNG

[redacted] committed on Dec 7, 2014 1 parent b0d5639

Showing 1 changed file with 26 additions and 28 deletions.

54 bitcoinjs-lib/src/jsbn/rng.js

```
@@ -8,15 +8,16 @@ var rng_state;
 8  8  var rng_pool;
 9  9  var rng_pptr;
10 10
11    - // Mix in a 32-bit integer into the pool
12    - function rng_seed_int(x) {
13    -     rng_pool[rng_pptr++] ^= x & 255;
14    -     rng_pool[rng_pptr++] ^= (x >> 8) & 255;
15    -     rng_pool[rng_pptr++] ^= (x >> 16) & 255;
16    -     rng_pool[rng_pptr++] ^= (x >> 24) & 255;
```